Міністерство освіти і науки України
Харківський Національний Університет Радіоелектроніки
Кафедра ПІ

Звіт
Практична робота №1
з дисципліни ”Методологія конструктивного мислення для наукових досліджень”

Виконав:
ст. гр. ІПЗм-26-3
Костюк Деніс Васильович

Перевірив:
доц. каф. ПІ
Каук Віктор Іванович

Харків 2026

ЗМІСТ

1 Хід роботи 3
1.1 Дослідницьке поле 3
1.2 Ключові проблеми 3
1.2.1 Методологія 3
1.2.2 Знайдені ключові джерела 4
1.2.3 Виявлені проблеми 5
1.3 Перевірені дослідницькі прогалини 6
1.4 Формування тем 8
1.4.1 Тема A (аналітична-теоритична) 8
1.4.2 Тема B (інженерна) 14
1.4.3 Тема C (міждисплінарна) 19
1.5 Порівняння тем 26
1.6 Peer review 28
1.6.1 Відгуки щодо теми A 28
1.6.2 Відгуки щодо теми B 28
1.6.3 Відгуки щодо теми C 28
1.7 Decision Matrix 28
Висновок 30

1 ХІД РОБОТИ

1.1 Дослідницьке поле

Дослідницьке поле роботи – дослідження продуктивності та надійності диспетчеризації віртуальних потоків на платформі JVM в умовах взаємодії з нативним кодом (FFM та JNI) і блокуючими операціями вводу-виводу. Англійською: Performance and scheduling reliability analysis of JVM virtual threads under native boundary interactions (FFM and JNI) and blocking I/O workloads.

Вибір поля обумовлено особистим інтересом до внутрішньої архітектури віртуальних машин, розробки високонавантажених серверних систем на Java/Kotlin та актуальністю переходу на віртуальні потоки (Project Loom) разом із новим інтерфейсом взаємодії з нативною пам'яттю (Panama FFM API).

Для обрання дослідницького поля виконувався пошук за темами, що відповідали моєму технічному стеку та сфері наукових інтересів (Java/Kotlin, багатопоточність, рантайми, системна оптимізація), по таких наукових репозиторіях і базах даних, як ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar, а також інженерних репозиторіях OpenJDK (JEPs). Джерела, використовувані надалі, було також знайдено через них.

1.2 Ключові проблеми

1.2.1 Методологія

Для формування бібліографічної бази та аналізу предметної області використовувався комбінований підхід: пошукова система Perplexity для первинної навігації та модульний булевий (бінарний) пошук у наукометричних базах (ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar). Перелік пошукових запитів:

- Базова поведінка диспетчера та масштабованість:
  - ”Virtual Threads” AND ”carrier starvation”
  - ”Project Loom” AND ”tail latency”
  - ”Virtual Threads” AND ”scalability”
- Нативний пінінг та міграція (JNI / FFM API):
  - ”Virtual Threads” AND ”carrier pinning” AND ”JNI”
  - ”Virtual Threads” AND ”FFM” AND ”pinning”
  - ”Foreign Function Memory” AND ”JNI” AND ”benchmark”

- Еволюція моніторів синхронізації (JEP 491 / JDK 24+):

– ”Virtual Threads” AND ”JEP 491”
– ”Virtual Threads” AND ”synchronized” AND ”pinning”
– ”residual pinning” AND ”JDK 24”

- Міжрівнева телеметрія та моделювання:

– ”Virtual Threads” AND ”eBPF” AND ”scheduler”
– ”Virtual Threads” AND ”queueing model”
– ”Virtual Threads” AND ”JFR” AND ”pinning”

1.2.2 Знайдені ключові джерела

1. FCoral: A Framework for Fine-grained Concurrency Evaluation Across Programming Language Runtimes. 2026. (https://doi.org/10.1145/3847668).
2. Misleading Microbenchmarks on the Java Virtual Machines. 2026. (https://doi.org/10.1145/3748522.3779882).
3. Practical Benchmarking Configurations for Reproducible Execution-Time Measurements of CI/CD-Style Workloads. 2026. (https://doi.org/10.1145/3838807).
4. Experimental Evaluation Methodology for the Era of No Steady Performance. 2026. (https://doi.org/10.1145/3839448).
5. Fray: An Efficient General-Purpose Concurrency Testing Platform for the JVM. 2025. (https://doi.org/10.1145/3764119).
6. Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads. 2024. (https://norma.ncirl.ie/8134/).
7. Virtual Threads after JDK 24: What Changed for Production Java. 2026. (https://www.infoq.com/articles/virtual-threads-after-jdk24/).
8. JEP 491: Synchronize Virtual Threads without Pinning. 2024. (https://openjdk.org/jeps/491).
9. Java Threading Options in 2026: Platform vs Virtual Threads. 2026. (https://www.jobrunr.io/en/blog/java-job-threading-options/).
10. Java 25 and the Evolution of Modern Java for Next-Generation Software Systems. 2026. (https://doi.org/10.55041/ijcope.v2i4.357).

11. eBPF jAgent: Transaction-level Resource Demand Tracing for Java Applications Using eBPF and OpenTelemetry. 2026. (`https://github.com/hm-green-it-lab/ssp2026`).

1.2.3 Виявлені проблеми

На основі аналізу наукової літератури, офіційних специфікацій OpenJDK та результатів інженерних досліджень виділено ключові проблеми диспетчеризації та надійності віртуальних потоків JVM:

1. **Залишкове блокування пулу потоків-носіїв на нативній межі (carrier-pool starvation).** Незважаючи на усунення моніторного пінінгу в JDK 24 (JEP 491), виконання нативного коду через C ABI (як у JNI, так і в Panama FFM API) та нативні зворотні виклики (callbacks) примусово утримує платформені потоки, спричиняючи приховане виснаження пулу ForkJoinPool при зростанні конкурентності.

2. **Відсутність замкнених аналітичних моделей масового обслуговування.** У науковій літературі бракує формалізованого математичного апарату теорії черг, який би дозволяв аналітично розраховувати критичний поріг насичення системи (λ_{crit}), довжину черг очікування та квантилі хвостових затримок для гібридних навантажень, що поєднують відмонтовувані операції вводу-виводу та невідмонтовувані нативні виклики.

3. **Ефект взаємного блокування задач та радіус ураження (blast radius).** Неізольоване закріплення потоків-носіїв під час тривалих нативних операцій блокує спільний пул ресурсів, що призводить до нелінійного падіння пропускної здатності та спалахів хвостових затримок (P_{99}) у сусідніх незалежних віртуальних потоках (зокрема неблокуючому мережевому I/O).

4. **Зміщення вузького місця та дефіцит архітектурних патернів ізоляції.** Неконтрольоване масштабування віртуальних потоків зміщує перевантаження на супутні підсистеми (пули з'єднань з базами даних, системні дескриптори), зумовлюючи каскадні відмови; водночас в інженерній практиці бракує верифікованих патернів захисту (семафорних бар'єрів-bulkheads, виділених пулів) для ізоляції нативного коду.

5. **Міжрівневий розрив спостережуваності рантайму та ядра ОС.** Планувальник JVM керує легковажними потоками у просторі користувача (M :

N), а операційна система – на рівні ядра (task_struct). Профайлери байткоду (JVMTI) не фіксують перемикання контексту ядра ОС (sched_switch, черги futex), тоді як системні трейсери (eBPF) прив'язані до потоків ядра та втрачають транзакційний контекст при виконанні віртуальних потоків.

6. **Ефект спостерігача та висока вартість багатофакторного тестування.** Глибоке системне трасування вносить значний оверхед у рантайм і спотворює вимірювання затримок, а повний емпіричний аналіз багатовимірного простору конфігурацій потребує надмірних ресурсів, що зумовлює потребу у легковажній телеметрії та сурогатних моделях машинного навчання.

1.3 Перевірені дослідницькі прогалини

Результати верифікації виявлених дослідницьких прогалин за правилом тріангуляції наведено в табл. 1.1.

Таблиця 1.1 – Верифікація дослідницьких прогалин та їх доказова база

Ідентифікатор	Формування	Опис та доведення
GEC-1, Енергія VT при пінінгу	На тлі декларованої енергоефективності віртуальних потоків на ізольованих задачах залишається недослідженим їхнє питоме енергоспоживання під час деградації супутніх пулів ресурсів та нативного пінінгу.	Неможливо довести за допомогою правила тріангуляції (у наявній доказовій базі менше 3 джерел із DOI; промислові звіти [Src-10], [Src-12] та специфікація [Src-11] не мають DOI). Підтверджуючі джерела з DOI: [Src-9] (DOI: `https://doi.org/10.1016/j.jss.2026.112963`) фіксує енергозбереження лише для чистих REST-профілів; [Src-1] (DOI: `https://doi.org/10.1145/3847668`) вимірює затримки віртуальних потоків без оцінки енергоспоживання.

Продовження таблиці 1.1

Ідентифікатор	Формування	Опис та доведення
GEC-2, Джиттер і планування на big.LITTLE	Теоретичний аналіз поширення часового джитеру в ланцюжках періодичних завдань та практичні алгоритми енергоефективного диспетчеризування на процесорах ARM big.LITTLE розвиваються ізольовано.	Доведено за правилом тріангуляції за трьома незалежними рецензованими джерелами з DOI: 1) моделі поширення джитеру конвеєрів абстраговані від планувальника ОС та енергії ([Src-15], DOI: https://doi.org/10.1109/RTSS66672.2025.00017); 2) алгоритми планування на big.LITTLE оптимізують дедлайни без контролю фазового джитеру ([Src-17], DOI: https://doi.org/10.1109/JIOT.2025.3568691; [Src-18], DOI: https://doi.org/10.1145/3716368.3735195); 3) системні огляди підтверджують метричний розрив між дедлайнами та енергією ([Src-16], DOI: https://doi.org/10.1016/j.sysarc.2025.103628).

Продовження таблиці 1.1

Ідентифікатор	Формування	Опис та доведення
GEC-3, Нативний бар'єр FFM/JNI у VT	Незважаючи на усунення моніторного пінінгу в JDK 24, поведінка нативного бар'єру FFM API у порівнянні з JNI та вводом-виводом залишається експериментально нерозчленованою в умовах насичення пулу носіїв.	Доведено за правилом триангуляції за трьома незалежними рецензованими джерелами з DOI: 1) наявні бенчмарки тестують рантайм як «чорну скриньку» або маскують деградацію пулу сокетами ([Src-1], DOI: `https://doi.org/10.1145/3847668`; [Src-8], DOI: `https://doi.org/10.7148/2025-0555`); 2) доведено нерепрезентативність ізольованих мікротестів нативного переходу через спотворення JIT-профілів ([Src-2], DOI: `https://doi.org/10.1145/3748522.3779882`); 3) доведено нестаціонарність фаз JVM, що спростовує наївні середні оцінки деградації пулу носіїв ([Src-4], DOI: `https://doi.org/10.1145/3839448`).

1.4 Формування тем

Для подальшого формування тем було обрано **GEC-3**. Вибір проводився за допомогою виключення. **GEC-1 –** відпала, бо недоводиться за правилом триангуляції. **GEC-2** непідходить через те що наявне устаткування SoC qualcom 425 – має архітектуру 4x Cortex-A53, яка є виключно енергоефктивною (LITTLE). Тому почнемо формувати тему навколо **GEC-3**.

1.4.1 Тема A (аналітична-теоритична)

Тема A є теоретико-аналітичним кандидатом дослідження, спрямованим на фундаментальну математичну формалізацію процесів диспетчеризації віртуальних

потоків JVM. Вона поєднує апарат класичної теорії масового обслуговування із системними механізмами ядра віртуальної машини HotSpot для аналітичного прогнозування деградації пулу потоків-носіїв на межі викликів FFM та JNI.

Заголовок (UA): Аналітичне моделювання та експериментальна верифікація блокування пулу потоків-носіїв віртуальних потоків на межі взаємодії FFM та JNI у різних версіях JVM.

Заголовок (EN): Analytical Modeling and Experimental Verification of Virtual Thread Carrier-Pool Pinning Across the FFM and JNI Boundary Under Stratified JVM Versions.

Дослідницьке поле: Комп'ютерні науки (спеціальність 122) та інженерія програмного забезпечення (спеціальність 121) – математичне та імітаційне моделювання складних обчислювальних систем, теорія масового обслуговування, стохастичний аналіз рантаймів системного програмного забезпечення.

Наукова проблема: Перехід від застарілого Java Native Interface (JNI) до нового Foreign Function and Memory (FFM) API змінює механіку взаємодії Java з нативним кодом, проте межа FFM/JNI залишається аналітично та експериментально недослідженою щодо ризиків голодування пулу потоків-носіїв (carrier-pool starvation) у моделі виконання віртуальних потоків ($M : N$). У той час як віртуальні потоки успішно відмонтовуються під час стандартного кооперативного блокування (неблокуючий мережевий I/O, `LockSupport.parkNanos`), нативні стек-фрейми C/C++ ABI примусово закріплюють потік-носій (carrier pinning). Це спричиняє нелінійну деградацію пропускної здатності та спалахи хвостових затримок, математичний зв'язок яких із кількістю носіїв, інтенсивністю надходження та структурою нативних викликів (downcalls і callbacks) досі формально не охарактеризовано.

Доказова база проблеми:

1. Нормативна специфікація OpenJDK JEP 491 ([Src-11]): декларує ліквідацію моніторного пінінгу в JDK 24, проте прямо констатує збереження нативного закріплення C ABI (як для JNI, так і для Panama FFM) та блокуючих callbacks у JDK 24/25.

2. Базові емпіричні межі віртуальних потоків Pandita ([Src-7]): зростання throughput на 60.79% та зниження затримки на 28.8% досягається лише для чистого I/O, тоді як вплив нативного бар'єру залишається нерозчленованим.

3. Хвостова деградація у крос-мовному фреймворку FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`): зафіксовано спалахи затримок P_{99} віртуальних потоків Java під час росту конкурентності, проте рантайм оцінюється як «чорна скринька» без ізоляції JNI/FFM.

4. Нестаціонарність рантайму JVM у дослідженні Antoch et al. ([Src-4], DOI: `https://doi.org/10.1145/3839448`): доведено відсутність стаціонарного стану в JVM через JIT-деоптимізацію та паузи GC, що свідчить про непридатність наївних стаціонарних оцінок без сегментації рядів.

Дослідницька прогалина: Класичні математичні моделі черг не підходять для віртуальних потоків Java: вони розглядають потоки або як повністю легковажні (що завжди звільняють процесор), або як важкі потоки ОС. У реальності виникає змішане навантаження: звичайні веб-запити легко вивільняють ядра процесора, а нативний С-код (JNI чи новий FFM API) примусово утримує їх, викликаючи раптовий колапс системи.

- **Математична модель змішаної черги:** створення формули розрахунку черг і затримок для гібридного навантаження, де частина потоків вільно відмонтовується, а частина – блокує ядра процесора.
- **Формула точки колапсу (λ_{crit}):** аналітичне виведення критичного порогу – за якої інтенсивності нативних викликів пул потоків-носіїв повністю вичерпується, а затримки (P_{99}) злітають у нескінченність.
- **Математичне порівняння технологій:** з'ясування, чи змінює заміна JNI на FFM та оновлення версії Java (JDK 21 проти JDK 25) теоретичну межу насичення системи.

Тип прогалини: Theoretical gap.

Об'єкт дослідження: Процес планування, відмонтування та закріплення потоків-носіїв віртуальних потоків при одночасному виконанні нативних (JNI/FFM) та керованих блокуючих операцій у віртуальній машині Java.

Предмет дослідження: Математичні моделі теорії масового обслуговування (багатокласові СМО з утриманням обслуговуючих приладів) та емпіричні розподіли затримок і пропускної здатності диспетчера `ForkJoinPool` на межі взаємодії JNI, FFM, callbacks та I/O за контрольованого паралелізму носіїв і версійної стратифікації JVM.

Дослідницькі питання (RQ):

• **RQ1:** Наскільки точно багатокласова модель теорії масового обслуговування здатна апроксимувати утилізацію пулу носіїв та час очікування в черзі, коли віртуальні потоки виконують суміш відмонтовуваних операцій, операцій із компенсацією потоків та невідмонтовуваних нативних фреймів (JNI/FFM downcalls та callbacks)?

• **RQ2:** Якою мірою архітектурний перехід від JNI до FFM зміщує аналітичний поріг вичерпання пулу носіїв (λ_{crit}) у міру масштабування тривалості блокування та конкурентності відносно фіксованого паралелізму носіїв?

• **RQ3:** Як міжверсійна стратифікація JVM (порівняння рантаймів JDK 21 та JDK 25) дозволяє аналітично ізолювати залишкове вузьке місце пінінгу нативних callbacks від застарілого пінінгу на моніторах synchronized?

Наукова гіпотеза: За однакової семантики задач та ресурсних обмежень час перебування задач у черзі спільного пулу носіїв підпорядковується закономірностям багатокласової системи масового обслуговування, де коефіцієнт завантаження системи визначається як сума парціальних навантажень:

$$\rho = \frac{\lambda_{\text{managed}} \cdot \bar{S}_{\text{managed}} + \lambda_{\text{native}} \cdot \bar{S}_{\text{native}}}{c},$$

де c – кількість потоків-носіїв. Фазовий перехід у режим колапсу затримок P_{99} настає при досягненні критичної інтенсивності λ_{crit}, коли частка закріплених нативних задач перевищує пропускну здатність носіїв ($\lambda_{\text{native}} \cdot \bar{S}_{\text{native}} \geq c$). Panama FFM скорочує накладні витрати виклику на мікросекундних тривалостях, проте для нативних callbacks обидва інтерфейси демонструють ідентичні асимптоти деградації незалежно від усунення моніторного пінінгу в JDK 24+.

Мета дослідження: Розробити та експериментально перевірити математичну модель черги, яка дозволяє розраховувати квантилі затримок і прогнозувати момент перевантаження системних потоків Java під час виклику нативного коду через JNI та FFM у різних версіях JVM.

Основні завдання:

1. Формалізувати стани переходів диспетчера віртуальних потоків JVM (`ForkJoinPool` carrier pool та механізм компенсації блокувань) в умовах кооперативного відмонтування, розширення пулу, моніторного та нативного пінінгу.

2. Вивести аналітичну модель черг (багатокласова СМО типу $M/G/c$ із

двома класами вимог – розвантажувані та блокуючі носій), що прогнозує межі пропускної здатності, довжину черги та квантилі затримок очікування.

3. Спроєктувати та реалізувати факторіальний контрольований експеримент із фіксацією семантики задач і ресурсних бюджетів, варіюючи версію JVM (JDK 21 проти JDK 25), нативний інтерфейс (JNI vs FFM), наявність callbacks, файловий/сокетний I/O та тривалість блокування.

4. Провести експериментальні прогони на фіксованій апаратній платформі з урахуванням усунення системного шуму та нестаціонарності JVM, виконати підгонку емпіричних кривих до теоретичної моделі та оцінити точність апроксимації.

Наукова новизна: Сформульовано першу емпірично верифіковану аналітичну модель масового обслуговування, яка математично ізолює внесок нативного закріплення стеків та зворотних викликів (для JNI та Panama FFM) від кооперативного розмонтування та моніторного пінінгу в розрізі версій JDK 21 та JDK 25.

Практичне значення: Отримання замкнених аналітичних формул та інженерного калькулятора критичного навантаження (λ_{crit}) для архітекторів високонавантажених систем, що дозволяє аналітично розраховувати максимально безпечне співвідношення нативних викликів до числа ядер CPU без потреби у тривалому емпіричному стрес-тестуванні для кожної нової конфігурації.

Методологія та методи дослідження: Аналітична теорія масового обслуговування (багатокласові системи черг із серверами утримання, метод додаткової змінної), чисельні методи оптимізації (Python, SymPy, NumPy, SciPy), лабораторний повнофакторний експеримент із мікробенчмаркінгом на базі JMH за методикою усунення JIT-викривлень ([Src-2]), низькооверхедна телеметрія HotSpot JVM через Java Flight Recorder (JFR) та HdrHistogram, а також методика оцінки нестаціонарних сегментів виконання ([Src-4]).

Базовий рівень (Baseline) та вибірка даних: Базовий рівень – віртуальні потоки OpenJDK 21 LTS із дефолтним числом носіїв, що виконують виключно чисто керовані блокування (`LockSupport.parkNanos`) та синтетичні неблокуючі нативні no-op виклики. Вибірка даних – первинні емпіричні ряди подій JFR (`jdk.VirtualThreadPinned`, черги `ForkJoinPool`) та розподіли затримок за факторіальною сіткою з не менше 72 унікальних конфігурацій (по 10

повторних прогонів для забезпечення статистичної потужності).

Метрики оцінки: Теоретичні: середня абсолютна відсоткова похибка (MAPE) та коефіцієнт детермінації (R^2) між аналітичним прогнозом і вимірюваннями. Емпіричні: пропускна здатність (ops/sec), затримки P_{50}, P_{95}, P_{99}, $P_{99.9}$ (мс), коефіцієнт утилізації носіїв та тривалість утримання carrier thread на операцію.

Інструменти та апаратне забезпечення: OpenJDK 21 LTS та 25 LTS, GCC/Clang, Make, JMH, JFR, HdrHistogram, Python 3 (SymPy, NumPy, SciPy, Matplotlib). Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux із фіксацією ядер через `taskset`.

Загрози валідності: Внутрішня – спотворення розподілу затримок через JIT-прогрів, паузи GC або нестаціонарність рантайму (мінімізація через warmup-ітерації та сегментацію рядів за [Src-4]); конструктивна – викривлення ізольованих JMH-тестів (мінімізація через фонове навантаження за [Src-2]); зовнішня – однопроцесорна архітектура x86_64 без урахування затримок міжвузлового доступу NUMA.

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – спрощена двокласова модель черги (відмонтовуване блокування проти нативного закріпленого), верифікована на JDK 21 та 25 для JNI та FFM при 3 рівнях паралелізму носіїв та 3 тривалостях блокування. Stretch Goal – розширення аналітичної моделі для врахування динамічного створення потоків-компенсаторів під час файлового I/O та ланцюжків Java $\rightarrow$ Native $\rightarrow$ Java $\rightarrow$ Native.

Доказова база прогалини: Аналітичний огляд [SGP-3] та картка [GEC-3] фіксують теоретичний розрив між класичними моделями систем масового обслуговування та реальною динамікою $M : N$ планувальників: рецензовані публікації або тестують рантайм емпірично як «чорну скриньку» ([Src-1]), або аналізують ізольовані мікротести без формалізації черг носіїв ([Src-2], [Src-4]), залишаючи взаємодію нативних і кооперативних класів задач без замкненого математичного апарату.

Критерій фальсифікованості: Виявлення вже опублікованої верифікованої математичної моделі черг для пінінгу нативних callbacks у FFM та JNI для JDK 24+ під контрольованим паралелізмом носіїв; або доведення того, що час очікування в черзі `ForkJoinPool` не піддається математичній

формалізації через недетермінізм планувальника ОС.

Обґрунтування вибору та ризики теми: Глибокий теоретичний фундамент (спеціальність 122), що поєднує класичну прикладну математику з системними механізмами ядра JVM, гарантує високу наукову новизну та публікабельність результатів у фахових виданнях. Головний ризик полягає у математичній складності: аналітичні рівняння багатокласових СМО для неекспоненційних розподілів часу блокування можуть стати нерозв'язними в замкненій формі, змушуючи зводити дослідження до чисельної імітації.

1.4.2 Тема B (інженерна)

Тема B є прикладним інженерним кандидатом дослідження, орієнтованим на вирішення практичних проблем стійкості (resilience) та продуктивності високонавантажених бекенд-систем на базі JVM. Вона зосереджена на емпіричному дослідженні взаємного впливу нативних викликів FFM/JNI та операцій вводу-виводу на спільний пул потоків-носіїв віртуальних потоків, а також на проєктуванні та експериментальній верифікації архітектурних патернів ізоляції ресурсів (bulkheading).

Заголовок (UA): Експериментальне дослідження продуктивності та розробка патернів ізоляції пулу носіїв віртуальних потоків при взаємодії через FFM та JNI під контрольованим навантаженням.

Заголовок (EN): Empirical Performance Evaluation and Carrier-Pool Isolation Patterns for Virtual Threads Across the FFM and JNI Boundary Under Controlled Pressure.

Дослідницьке поле: Інженерія програмного забезпечення (спеціальність 121), архітектура програмних систем, високопродуктивні обчислення на платформі JVM, патерни надійності та стійкості (Resilience & Bulkheading).

Наукова проблема: Інженерні команди, що мігрують високонавантажені бекенд-сервіси на віртуальні потоки та планують заміну застарілого JNI на Panama FFM API, позбавлені систематичних емпіричних настанов щодо того, як нативні виклики (downcalls), зворотні виклики (callbacks) та змішані операції вводу-виводу взаємодіють під навантаженням. Неізольований нативний пінінг здатен приховано паралізувати сусідні кооперативні сокетні чи файлові операції, що ділять спільний пул потоків-носіїв.

Доказова база проблеми:

1. У бенчмарках JobRunr ([Src-12]) неконтрольований пінінг потоків-носіїв обвалював throughput у 14 разів (з 462 до 33 jobs/s) та спричиняв вибух затримок P_{99} у 27 разів (з 253 мс до 6799 мс, тобто до 6.8 секунд).

2. У дослідженнях InfoQ ([Src-10]) продемонстровано явище «зміщення вузького місця» (bottleneck relocation): масштабування віртуальних потоків без обмеження пулу перенасичує пул з'єднань БД (HikariCP), викликаючи близько 37% помилок (36.91% vs 37.08%) через відсутність архітектурних перегородок (bulkheads); також підтверджено, що після усунення моніторного пінінгу в JDK 24 нативні виклики (JNI/FFM) залишаються головним джерелом закріплення носіїв.

3. Офіційні застереження Quarkus та OpenJDK щодо блокуючих зворотних викликів (native callbacks) із C/C++ у Java, які повністю блокують потоки-носії.

4. В емпіричному огляді платформи Java 25 ([Src-24]) зафіксовано, що чисті накладні витрати виклику FFM API зрівнялися з JNI (відхилення в межах 1.8% для пакетних навантажень), що доводить: системна деградація при взаємодії з нативним кодом спричинена не затримкою диспетчеризації виклику, а саме неізольованим закріпленням потоків-носіїв під час блокування.

Дослідницька прогалина: Віртуальні потоки створені для легкого масштабування, проте виклики нативного C-коду (через JNI або новий FFM API) блокують спільні потоки процесора і зупиняють роботу сусідніх веб-запитів. Спроби вирішити це класичним збільшенням пулу системних потоків призводять до надмірної перевитрати оперативної пам'яті.

• **Межа стійкості планувальника:** за якого співвідношення інтенсивності вхідного навантаження, кількості потоків-носіїв та тривалості нативних операцій (T_{native}) відбувається перехід від стабільної роботи до лавиноподібного зростання хвостових затримок (P_{99}) для JNI та Panama FFM.

• **Ефективність патернів ізоляції:** наскільки результативно прикладні механізми ізоляції (обмеження паралелізму через семафори за патерном Bulkhead або винесення нативних викликів у виділені пули виконавців) запобігають вичерпанню `ForkJoinPool` при мінімальних накладних витратах на пам'ять і додаткову затримку.

Тип прогалини: Empirical gap.

Об'єкт дослідження: Високонавантажені програмні системи на базі

JVM, що одночасно виконують змішані нативні (JNI/FFM) та мережеві/файлові I/O-bound задачі у віртуальних потоках.

Предмет дослідження: Емпіричні профілі деградації пропускної здатності та квантилів затримок P_{95}, P_{99}, $P_{99.9}$, а також архітектурні патерни запобігання голодуванню пулу носіїв (семафорні перегородки-bulkheads, виділені пули платформних потоків для нативних викликів, тюнінг паралелізму планувальника).

Дослідницькі питання (RQ):

• **RQ1:** За однакової семантики коду та ресурсних обмежень, як співвідносяться JNI та FFM за пропускною здатністю, накладними витратами процесора та хвостовою затримкою при масштабуванні тривалості нативного блокування, конкурентності та глибини нативних callbacks?

• **RQ2:** Наскільки сильно закріплені нативні виклики JNI та FFM погіршують роботу сусідніх віртуальних потоків, які виконують неблокуючий сокетний I/O на спільному пулі носіїв у різних версіях JVM (JDK 21 проти JDK 25)?

• **RQ3:** Чи здатні легковажні прикладні програмні обгортки (адаптивні семафорні бар'єри або пули розвантаження перед переходом у нативний код) гарантувати ізоляцію пулу носіїв із меншими витратами пам'яті, ніж глобальне завищення розміру `maxPoolSize` планувальника?

Наукова гіпотеза: Panama FFM забезпечує статистично нижчий овергед виклику порівняно з JNI для мікросекундних операцій, проте за тривалого блокування та наявності нативних callbacks обидва інтерфейси викликають однаковий катастрофічний ріст затримок P_{99} у сусідніх сокетних потоках, якщо кількість задач перевищує паралелізм носіїв; застосування обмеженого семафорного бар'єра (Semaphore-guarded bulkhead) на рівні прикладного шару відновлює затримку P_{99} сокетного I/O до рівня ізольованого виконання, споживаючи менше пам'яті, ніж надмірне масштабування carrier pool.

Мета дослідження: Розробка відтворюваного багатофакторного бенчмаркінгового комплексу для визначення точних експлуатаційних меж відмови JNI та FFM під навантаженням carrier-пулу, та створення верифікованої програмної бібліотеки патернів ізоляції для запобігання взаємному блокуванню завдань.

Основні завдання:

1. Розробити автоматизований тестовий стенд на Java та C, що моделює

еквівалентні сценарії для JNI, FFM, callbacks (виклик блокуючих Java-методів із C), файлового I/O, loopback-сокетного I/O та сліпу.

2. Провести систематичний багатофакторний експеримент за матрицею версій JVM (JDK 21 LTS vs. JDK 25 LTS), розмірів пулу носіїв, конкурентності та тривалості нативних задач.

3. Кількісно виміряти радіус ураження (cross-workload interference / blast radius) нативного пінінгу на сусідні сокетні віртуальні потоки.

4. Реалізувати та експериментально порівняти інженерні патерни захисту (семафорні обгортки, виділені пули платформних потоків для нативного коду, адаптивний тюнінг) і сформувати практичне дерево прийняття рішень для розробників.

Наукова новизна: Сформовано комплексну порівняльну характеристику взаємного впливу JNI, FFM, нативних callbacks та сокетного/файлового I/O в розрізі версій JDK 21 та JDK 25, поєднану з кількісною оцінкою ефективності архітектурних патернів ізоляції нативного бар'єру за фіксованого бюджету пам'яті та ядер CPU.

Практичне значення: Створено відкритий модульний тестовий стенд (benchmark harness), готову до впровадження легковажну Java-бібліотеку патернів ізоляції віртуальних потоків від нативного пінінгу, а також інженерну матрицю рішень, яка вказує, коли FFM можна викликати безпосередньо з віртуального потоку, а коли потрібна обов'язкова ізоляція чи делегування.

Методологія та методи дослідження: Експериментальна інженерія програмного забезпечення: контрольоване лабораторне бенчмаркування (JMH для мікротестів, кастомний генератор закритого/відкритого циклу навантаження для змішаних сценаріїв), низькооверхедне профілювання через JFR та `async-profiler`, порівняльний архітектурний аналіз патернів.

Базовий рівень (Baseline) та вибірка даних: Базовий рівень – стандартне неізольоване виконання у віртуальних потоках за замовчуванням (`Executors.newVirtualThreadPerTaskExecutor()`) із дефолтним числом потоків-носіїв без використання патернів ізоляції чи пулів розвантаження. Вибірка даних – логи часових рядів затримок HdrHistogram, профілі процесора та алокацій, дампи трас JFR-подій пінінгу та заміри використання оперативної пам'яті (Heap і RSS).

Метрики оцінки: Пропускна здатність (ops/sec), хвостова затримка (P_{95}, P_{99}, $P_{99.9}$, $P_{99.99}$ у мс), коефіцієнт ампліфікації затримки сокетних операцій ($L_{\text{ampl}} = \frac{P99_{\text{with_native}}}{P99_{\text{isolated}}}$), споживання пам'яті (Heap/RSS у МБ) та частота подій `jdk.VirtualThreadPinned`.

Інструменти та апаратне забезпечення: OpenJDK 21 та 24/25, GCC/Clang, JMH, HdrHistogram, async-profiler, Docker, bash/Python для автоматизації тестів. Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux (4 ядра виділяються під carrier pool через `taskset`, 2 ядра – під локальний генератор навантаження та збір метрик).

Етичні ризики: Відсутні. Дослідження використовує виключно синтетичний відкритий тестовий код і вільні інструменти без залучення комерційних чи персональних даних.

Загрози валідності: Внутрішня – координоване опущення (Coordinated Omission) у генераторі навантаження (мінімізація через використання HdrHistogram із компенсацією затримок або генерацію запитів за відкритим циклом); конструктивна – вплив системного кешу сторінок ОС на файловий I/O (мінімізація: тестування `tmpfs` проти синхронного запису `O_DIRECT`); зовнішня – складність структур пам'яті при маршалінгу в FFM варіюється в реальних бібліотеках (мінімізація: стандартизація корисного навантаження у C-бібліотеці).

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – повноцінний бенчмаркінговий стенд за всіма 8 ключовими факторами (версії JVM, JNI vs FFM, callbacks, сокети, файли, sleep, паралелізм носіїв, тривалість блокування), порівняльні таблиці меж деградації та оцінка базового патерну ізоляції (семафорний bulkhead проти дефолту). Stretch Goal – створення реактивного адаптивного обмежувача конкурентності на базі API `jdk.jfr.consumer.RecordingStream`, який у реальному часі відстежує частоту та тривалість подій `jdk.VirtualThreadPinned` і автоматично регулює кількість дозволів семафора для утримання P_{99} затримки в межах SLA.

Ключові верифіковані джерела: JEP 491 ([Src-11]), FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`), Schiavio et al. ([Src-2], DOI: `https://doi.org/10.1145/3748522.3779882`), Stanojević et al. ([Src-3], DOI: `https://doi.org/10.1145/3838807`), JobRunr ([Src-12]), InfoQ ([Src-10]) та огляд Java 25 ([Src-24], DOI: `https://doi.org/10.34726/hss.2025.`

102537).

Доказова база прогалини: Аналітичний огляд [SGP-3] та картка [GEC-3] підтверджують метричний та інженерний розрив: індустріальні звіти фіксують катастрофічні наслідки пінінгу ([Src-12]) та релокації вузьких місць ([Src-10]), а специфікація JEP 491 ([Src-11]) констатує непереборність нативного закріплення; проте в літературі відсутня кількісна оцінка взаємного блокування задач на спільному пулі носіїв та верифікована оцінка компромісу між патернами bulkheading і пам'яттю системи.

Критерій фальсифікованості: Публікація тотожного відкритого бенчмаркінгового дослідження з повним факторним покриттям (FFM, JNI, callbacks, сокети, версії JDK 21/25) та готовою оцінкою патернів ізоляції до завершення роботи; або доведення того, що накладні витрати будь-якого прикладного патерну ізоляції (семафора) повністю нівелюють перевагу віртуальних потоків.

Обґрунтування вибору та ризики теми: Найвища практична інженерна цінність для індустрії (спеціальність 121 – Інженерія програмного забезпечення), створення конкретних артефактів (open-source бібліотека, бенчмарк, матриця рішень), найвища реалістичність виконання (92/100) за 2–3 місяці на наявному комп'ютері. Головний ризик: більш прикладний інженерний фокус із меншою теоретичною новизною (менше математичного апарату порівняно з аналітичним моделюванням черг); необхідність акуратного контролю тривалості багатофакторних прогонів для запобігання перегріву чи часових затримок експерименту.

1.4.3 Тема C (міждисплінарна)

Тема C є міждисциплінарним frontier-кандидатом дослідження, що розташовується на перетині системного інжинірингу рантаймів, низькорівневої діагностики ядра операційної системи та прикладного аналізу даних. Вона присвячена подоланню розриву спостережуваності між простором користувача JVM та ядром Linux за допомогою синхронізованої телеметрії (JFR + eBPF/perf), а також побудові швидких сурогатних моделей машинного навчання для передбачення нелінійних точок колапсу продуктивності віртуальних потоків на багатовимірному просторі параметрів.

Заголовок (UA): Міжрівнева діагностика та сурогатне моделювання станів блокування віртуальних потоків на межі JVM та ядра ОС при виконанні нативних викликів FFM і JNI.

Заголовок (EN): Cross-Layer Observability and Surrogate Performance Modeling of Virtual Thread Pinning Across the JVM and OS Kernel Boundary in FFM and JNI Workloads.

Дослідницьке поле: Комп'ютерні науки / Інформаційні системи та технології (спеціальності 122 / 126), Data Science & Systems Observability, низькорівнева діагностика ядра ОС (Linux kernel & eBPF tracing), сурогатне машинне навчання у системному аналізі продуктивності.

Наукова проблема: Пінінг віртуальних потоків на межі FFM/JNI є складним міжрівневим феноменом: планувальник JVM керує відображенням віртуальних потоків на потоки-носії у просторі користувача (user space), тоді як нативні виклики (downcalls), зворотні виклики (callbacks) та змішані файлові/мережеві операції переходять у некеровані C-фрейми та системні виклики ядра ОС (`futex`, `epoll_wait`, `read`/`write`, `nanosleep`). Аналіз цієї межі виключно через метрики JVM або виключно через метрики ОС приховує механіку взаємодії нативних фреймів із планувальником ядра в умовах насичення пулу носіїв. Крім того, повне вичерпне бенчмаркування 8-вимірного комбінаторного простору конфігурацій є обчислювально затратним.

Доказова база проблеми:

1. Нормативна специфікація OpenJDK JEP 491 ([Src-11]): офіційно підтверджує, що ліквідація моніторного пінінгу в JDK 24 не поширюється на нативний код: виклики через JNI та Panama FFM при переході через системний бар'єр C ABI у ядро ОС примусово утримують платформні потоки-носії.

2. Обмеження існуючих інструментів тестування конкурентності JVM (Fray, ACM OOPSLA 2025, [Src-6], DOI: `https://doi.org/10.1145/3764119`): сучасні платформи детермінованого тестування багатопоточності HotSpot діють виключно на рівні байткоду/JVMTI через shadow locks і принципово не відстежують стани перемикання контексту ядра Linux (`sched_switch`, черги очікування на `futex`), що приховує причини міжрівневих блокувань.

3. Нестаціонарність та фазові переходи рантайму JVM (Antoch et al., ACM OOPSLA 2026, [Src-4], DOI: `https://doi.org/10.1145/3839448`):

доведено, що сучасні навантаження на JVM не досягають стаціонарного стану через фонову деоптимізацію та перебудову JIT-профілів, що зумовлює нелінійну природу деградації та вимагає використання нелінійних моделей машинного навчання (GBDT).

4. Емпіричні свідчення виснаження носіїв у промислових бенчмарках (JobRunr, [Src-12]; InfoQ, [Src-10]): JobRunr зафіксував падіння throughput у 14 разів (з 462 до 33 jobs/s) та зліт хвостових затримок P_{99} у 27 разів (з 253 мс до 6799 мс) під час пінінгу носія; InfoQ довів явище зміщення вузького місця (bottleneck relocation) та виникнення $\sim$ 37% помилок при неізольованому масштабуванні віртуальних потоків.

5. Обмеження сучасного міжрівневого eBPF-трасування JVM (Liegsalz & Brunnert, SSP 2026, [Src-25]): у дослідженні трасування Java через Linux eBPF автори визнають, що їхня модель прив'язана до задач ядра (`task_struct`) і повністю втрачає кореляцію при переході на віртуальні потоки; крім того, трасування через DTrace USDT-проби збільшує затримку у 6.1 раза (+507%) і спотворює CPU time у 7.4 раза через оверхед пасток переходу в ядро, що обґрунтовує потребу у низькооверхедному конвеєрі JFR + eBPF kernel tracepoints.

Дослідницька прогалина: Блокування віртуальних потоків на нативному коді створює міжрівневу «сліпу зону». Профайлери Java не бачать, що відбувається всередині ядра Linux (блокування системних викликів і м'ютексів), а системні утиліти ядра (eBPF) бачать лише процесор і втрачають контекст віртуальних потоків. При цьому спроби детального системного трасування сповільнюють програму на 500% (ефект спостерігача), а перевірити всі комбінації налаштувань вручну – нереально довго.

• **Міжрівнева телеметрія (Java + Linux):** як поєднати легкі логи Java (JFR) із лічильниками ядра Linux без падіння швидкодії, щоб точно діагностувати причину зависання потоків у системних викликах ядра.

• **ML-прогнозування збоїв (сурогатна модель):** як за допомогою машинного навчання на основі невеликої вибірки тестів заздалегідь передбачати момент колапсу затримок (P_{99}) для невідомих налаштувань сервера без тривалого вичерпного тестування.

Тип прогалини: Methodological gap.

Об'єкт дослідження: Міжрівневий життєвий цикл виконання гібридних

Java/нативних навантажень, що охоплює планувальник віртуальних потоків JVM, інтерфейси нативної взаємодії JNI/FFM та планувальник задач ядра операційної системи Linux.

Предмет дослідження: Синхронізована міжрівнева телеметрія (поєднання подій JVM Flight Recorder із лічильниками ядра Linux `perf`/eBPF) та статистичні сурогатні моделі машинного навчання (Response Surface Methodology, GBDT – LightGBM/XGBoost, SHAP-аналіз взаємодій факторів) для виявлення та прогнозування колапсу продуктивності (хвостових затримок P_{99}).

Дослідницькі питання (RQ):

• **RQ1:** Які характерні сигнатури планування в ядрі ОС (частота перемикання контексту `sched_switch`, частка перебування в режимі ядра, блокування на `futex`, час очікування в черзі планувальника) розрізняють пінінг нативних фреймів JNI та FFM від кооперативного зняття віртуальних потоків (unmounting) та блокувань сокетного/файлового I/O в різних версіях JVM?

• **RQ2:** Як зворотні нативні виклики (Java $\rightarrow$ C $\rightarrow$ блокуючий Java-код) проявляються у комбінованій телеметрії JVM та ядра ОС у версіях, де усунуто пінінг на моніторах `synchronized` (JDK 24+ / JEP 491), порівняно з легасі-версіями (JDK 21)?

• **RQ3:** Чи здатна інтерпретована сурогатна модель машинного навчання, навчена на вибірці простору параметрів за методом латинського гіперкуба (Latin Hypercube Sampling / дробовий факторний план), з високою точністю ($R^2 > 0.85$, $F_1 > 0.9$) прогнозувати настання нелінійного колапсу затримок P_{99} та голодування пулу носіїв на невідомих конфігураціях?

Наукова гіпотеза: Поєднання низькорівневих метрик стану потоків ядра ОС (tracepoints ядра, апаратні лічильники `perf`) із JFR-подіями пінінгу JVM дозволяє ідентифікувати межі експлуатаційних режимів (наприклад, приховану конкуренцію за нативні м'ютекси ядра проти кооперативного розвантаження через `epoll`), які неможливо розрізнити лише за метриками простору користувача JVM; крім того, ансамблеві деревні сурогатні моделі (LightGBM/XGBoost із SHAP-атрибуцією) здатні передбачати хвостову деградацію P_{99} та виявляти домінантні міжфакторні взаємодії (зокрема 3-факторну взаємодію «наявність callbacks $\times$ тривалість блокування $\times$ співвідношення конкурентності до розміру пулу носіїв») на основі дробової вибірки ($< 20\%$ від повного комбінаторного

простору).

Мета дослідження: Побудова синхронізованого програмного конвеєра міжрівневої телеметрії (JVM-to-OS) та навчання інтерпретованої сурогатної моделі для діагностики, аналізу факторної чутливості та швидкого прогнозування нелінійного колапсу продуктивності віртуальних потоків на межі FFM/JNI під багатовимірним навантаженням.

Основні завдання:

1. Розробити синхронізований вимірювальний комплекс, що зіставляє події JVM JFR (`jdk.VirtualThreadPinned`, `jdk.ThreadPark`) із точками трасування ядра Linux (`sched:sched_switch`, системними викликами `syscalls:sys_enter_*`, лічильниками `perf_event_open`/`bpftrace`) для JNI, FFM, callbacks, сокетів, файлів та сліпу.

2. Спроєктувати та реалізувати план багатофакторного експерименту (поєднання повного факторного ядра з вибіркою за латинським гіперкубом LHS для неперервних параметрів) у розрізі версій JDK 21 та JDK 25 за фіксованого бюджету ядер та пам'яті.

3. Провести дисперсійний та причинно-наслідковий аналіз чутливості (ANOVA, індекси Соболя, значення SHAP) для кількісного ранжування внеску кожного з 8 факторів та їхніх взаємодій у деградацію квантилів затримок.

4. Навчити, валідувати та порівняти сурогатні моделі (поліноміальні моделі поверхні відгуку RSM та градієнтний бустинг LightGBM/XGBoost) для прогнозування пропускної здатності та кліфів P_{99} на відкладених (out-of-distribution) тестових конфігураціях.

5. Розробити прототип легкої утиліти автоматизованої експрес-діагностики («Loom Cross-Layer Advisor»), яка аналізує короткий комбінований профіль та оцінює дистанцію до режиму голодування носіїв.

Наукова новизна: Міждисциплінарний синтез методів системного інжинірингу рантаймів, низькорівневого трасування ядра Linux та сурогатного машинного навчання, що вперше забезпечує кількісну оцінку міжфакторної взаємодії 8 параметрів на межі JNI/FFM та розкриває механізми поведінки ядра ОС при блокуванні віртуальних потоків на спільних потоках-носіях.

Практичне значення:

• Відкритий інструментальний скрипт міжрівневого моніторингу

(eBPF + JFR) для низькооверхедної діагностики виробничих затримок у Linux-середовищах.

• Опублікований відкритий табличний датасет багатофакторних прогонів для спільноти дослідників продуктивності JVM.

• Легковажна сурогатна модель та утиліта експрес-оцінки конфігурації для автоматизованого тюнінгу параметрів пулу носіїв у CI/CD або рантаймі без проведення тривалого вичерпного тестування.

Методологія та методи дослідження: Теорія планування експерименту (Design of Experiments – DoE: Full Factorial + Latin Hypercube Sampling); системне низькорівневе трасування ядра Linux (`perf`, `bpftrace`, kernel tracepoints); дисперсійний аналіз чутливості (індекси Соболя, ANOVA); машинне навчання з учителем (Response Surface Methodology, GBDT: LightGBM / XGBoost) із k-кратною крос-валідацією та інтерпретацією через адитивні пояснення Шеплі (SHAP values).

Базовий рівень (Baseline) та вибірка даних: Однорівневі регресійні евристики та лінійні моделі виключно на базі метрик простору користувача JVM (без системних лічильників ядра), а також емпіричні базові профілі JDK 21 для JNI/FFM без розділення режимів блокування ядра та кооперативного перемикання. Вибірка даних – публічний багатовимірний табличний датасет експериментальних запусків (кілька тисяч конфігураційних точок), де кожен запис містить 8 незалежних змінних (факторів), набір проміжних міжрівневих метрик (JFR-події, лічильники ядра ОС, перемикання контексту) та цільові показники (throughput, квантилі затримок P_{50}–$P_{99.9}$, бінарна мітка starvation cliff).

Метрики оцінки: Якість сурогатних моделей: коефіцієнт детермінації (R^2), RMSE, MAPE для логарифма затримки $\ln(P_{99})$, Precision, Recall та F_1-score для класифікації настання кліфу голодування. Факторна чутливість: індекси Соболя першого та повного порядку, абсолютні значення SHAP. Системні міжрівневі метрики: швидкість добровільних/примусових перемикань контексту (`voluntary_ctxt_switches`, `nonvoluntary_ctxt_switches` / sec), співвідношення Kernel/User CPU time, середня тривалість блокування на `futex`, частота подій `jdk.VirtualThreadPinned`.

Інструменти та апаратне забезпечення: OpenJDK 21 та 24/25 (HotSpot JVM), GCC/Clang, Make, Linux `perf`, `bpftrace`, BCC / eBPF kernel tracepoints, Python 3.11+ (Pandas, SciPy, Scikit-learn, XGBoost, LightGBM, SHAP, SALib,

Matplotlib, Seaborn), JMH та Java Flight Recorder (JFR). Апаратна платформа: Intel Core i5-10400F (6 ядер / 12 потоків), 32 GB RAM, ОС Linux. Навчання табличних ML-моделей на вибірках у кілька тисяч рядків займає менше 60 секунд на 6 ядрах.

Етичні ризики: Відсутні. Дослідження спирається на синтетичні навантаження, відкриті системні інтерфейси та ліцензійно чисті компоненти без персональних даних або комерційної таємниці.

Загрози валідності: Внутрішня (Observer effect) – накладні витрати одночасного трасування JFR та ядра Linux можуть спотворити вимірювання (мінімізація: внутрішньоядерні eBPF map-агрегації та лічильники `perf stat` замість дискового дампу кожного виклику з фіксацією оверхеду $< 2\%$); конструктивна – перенавчання сурогатної моделі (мінімізація: тестування на відкладених конфігураціях з інтерполяцією та екстраполяцією); зовнішня – специфіка ядра Linux та планувальника CFS/EEVDF (мінімізація: фіксація версії ядра Linux 6.8+ та конфігурації у вихідних даних).

Мінімальне життєздатне дослідження (MVR) та Stretch Goal: MVR – повний емпіричний датасет за 8 факторами, зіставлений із лічильниками `perf stat` та ключовими подіями ядра, розрахунок глобальної чутливості факторів через SHAP, та валідована сурогатна модель (LightGBM/XGBoost), що передбачає настання P_{99} starvation cliff із точністю $F_1 > 0.85$. Stretch Goal – розробка компактної консольної утиліти («Loom Cross-Layer Advisor»), яка знімає короткий 10-секундний комбінований профіль (JFR + `perf`) працюючого застосунку і на основі навченої сурогатної моделі сигналізує, якщо поточна інтенсивність нативних викликів перебуває ближче ніж за 15% до точки колапсу продуктивності carrier-пулу.

Ключові верифіковані джерела: JEP 491 ([Src-11]), FCoral ([Src-1], DOI: `https://doi.org/10.1145/3847668`), Antoch et al. ([Src-4], DOI: `https://doi.org/10.1145/3839448`), Fray ([Src-6], DOI: `https://doi.org/10.1145/3764119`), Schiavio et al. ([Src-2], DOI: `https://doi.org/10.1145/3748522.3779882`), InfoQ ([Src-10]), JobRunr ([Src-12]) та дослідження eBPF jAgent Liegsalz & Brunnert ([Src-25]).

Доказова база прогалини: Аналітичні огляди [SGP-3] та картка [GEC-3] фіксують гострий міжрівневий розрив спостережуваності: провідні платформи тестування байткоду ([Src-6]) сліпі до станів ядра ОС, тоді як новітні

eBPF-конвеєри ([Src-25]) офіційно визнають втрату кореляції з віртуальними потоками та колосальний оверхед перехоплення (+507%), підтверджуючи відсутність легковажних міжрівневих методів предиктивної діагностики голодування носіїв.

Критерій фальсифікованості: Якщо експериментально виявиться, що багатофакторні взаємодії між JNI/FFM, callbacks, сокетами та розміром пулу носіїв є абсолютно лінійними/адитивними і не мають нелінійних точок перегину (cliffs), що робить застосування нелінійних сурогатних моделей надлишковим; або якщо до моменту захисту буде опубліковано аналогічне міжрівневе дослідження з комбінованим eBPF+JFR аналізом та сурогатним моделюванням межі віртуальних потоків для FFM/JNI.

Обґрунтування вибору та ризики теми: Ідеальний міждисциплінарний науковий профіль на стику системного програмування та аналізу даних (спеціальності 122 «Комп'ютерні науки» або 126 «Інформаційні системи та технології»). Забезпечує високу наукову новизну (публікабельність у Scopus/WoS), відкритий цінний датасет і прикладну утиліту, демонструючи володіння як низькорівневим системним стеком Linux, так і сучасними методами Data Science / Machine Learning. Головний ризик полягає у вимозі до подвійної експертизи (double skill curve): дослідник повинен одночасно володіти низькорівневим інструментарієм ядра Linux (C, `perf`, eBPF/`bpftrace`) та стеком машинного навчання (Python, Scikit-learn, XGBoost, SHAP, Sobol analysis), що вимагає суворого контролю часових витрат.

1.5 Порівняння тем

Зведене порівняння трьох кандидатів магістерських тем дослідження (A, B та C) за ключовими класифікаційними та методологічними критеріями наведено в табл. 1.2.

Таблиця 1.2 – Порівняльна таблиця тем A, B і C

Критерій	Тема A	Тема B	Тема C
Тип теми	Аналітична / теоретична	Прикладна / інженерна	Міждисциплінарна / frontier

Продовження таблиці 1.2

Критерій	Тема A	Тема B	Тема C
Тип gap	Theoretical gap	Empirical gap	Methodological gap
Метод	Теорія масового обслуговування (багатокласові СМО $M/G/c$), чисельна оптимізація (SymPy), бенчмаркінг на базі JMH	Експериментальне бенчмаркування (JMH, генератор навантаження), профілювання (JFR, `async-profiler`), патерни bulkheading	Планування експерименту (DoE, LHS), системне трасування ядра Linux (eBPF, `perf`), аналіз чутливості (Sobol, SHAP), GBDT (LightGBM/ XGBoost)
Головна метрика	Похибка MAPE, R^2 апроксимації черги, хвостова затримка P_{99}, поріг λ_{crit}	Пропускна здатність (ops/s), затримка P_{99}, коефіцієнт ампліфікації сокетів L_{ampl}, пам'ять (Heap/RSS)	Метрики ML (R^2, RMSE, F_1-score для starvation cliff), значення SHAP, перемикання контексту (`sched_switch`)
Головний результат	Аналітична модель черги (дворівнева СМО), замкнені формули точки насичення λ_{crit} та ймовірності голодування $P_{\text{starvation}}$, програмний калькулятор	Відкритий 8-факторний бенчмаркінговий стенд, оцінка blast radius, Java-бібліотека патернів ізоляції (Resilience Library), матриця інженерних рішень	Синхронізований конвеєр телеметрії (JFR + eBPF/`perf`), публічний датасет, сурогатна модель прогнозування P_{99}, утиліта «Loom Cross-Layer Advisor»

Продовження таблиці 1.2

Критерій	Тема A	Тема B	Тема C
Складність стенда	Низька (локальний стенд JMH/JFR на 72 конфігурації для емпіричної валідації аналітичної моделі)	Середня (модульний автоматизований стенд Java/C зі змішаним навантаженням та реалізацією патернів bulkheading)	Висока (міжрівневий конвеєр трасування ядра Linux через eBPF/`perf` + HotSpot JFR із мінімізацією оверхеду $< 2\%$ та ML-пайплайн)
Головний ризик	Аналітична нерозв'язність рівнянь багатокласової СМО в замкненій формі для неекспоненційних розподілів часу блокування	Менша теоретична новизна (більш прикладний фокус); ризик перегріву та троттлінгу CPU через тривалість багатофакторних запусків	Вимога подвійної експертизи (double skill curve: Linux kernel/eBPF та Data Science/ML); ризик розмивання фокусу між трасуванням і моделюванням

1.6 Peer review

1.6.1 Відгуки щодо теми A

1.6.2 Відгуки щодо теми B

1.6.3 Відгуки щодо теми C

1.7 Decision Matrix

ВИСНОВОК